

МФТИ

Алгоритмы и структуры данных II

## **Семинар 01**

Динамическое программирование (1)

Весна 2026

Строгие подробные решения с доказательствами

## Содержание

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| <b>Обозначения и соглашения</b>                                             | <b>2</b>  |
| <b>1 Задача 1. Игра на концах массива</b>                                   | <b>3</b>  |
| <b>2 Задача 2. Путь с минимальным числом завершающих нулей</b>              | <b>4</b>  |
| <b>3 Задача 3. Удаление внутренних элементов</b>                            | <b>6</b>  |
| <b>4 Задача 4. Игра с камнями</b>                                           | <b>7</b>  |
| <b>5 Задача 5. Гладкие десятичные числа</b>                                 | <b>8</b>  |
| <b>6 Задача 6. Максимальная цепочка вложенных кругов</b>                    | <b>9</b>  |
| <b>7 Задача 7. Разбиение скобочной строки</b>                               | <b>10</b> |
| <b>8 Задача 8. Умножение соседних чисел на <math>-1</math></b>              | <b>11</b> |
| <b>9 Задача 9. Разложения целого числа</b>                                  | <b>12</b> |
| <b>10 Задача 10. Максимальный квадрат из единиц</b>                         | <b>13</b> |
| <b>11 Задача 11. Удаление каждого <math>k</math>-го элемента по раундам</b> | <b>14</b> |
| <b>12 Задача 12. Теорема Эрдёша-Секереша</b>                                | <b>15</b> |

## Обозначения и соглашения

Во всех оценках  $n$  обозначает размер входа, если в условии конкретной задачи не сказано иначе. Значение  $+\infty$  используется для недостижимого состояния в задачах минимизации, а  $-\infty$  — в задачах максимизации. Слово «путь» может означать маршрут с повторениями вершин только там, где это прямо следует из постановки; во всех остальных случаях путь простой.

## 1. Задача 1. Игра на концах массива

**Условие.** Два игрока по очереди забирают левый или правый элемент массива  $a_1, \dots, a_n$ . Выигрыш игрока равен сумме забранных им чисел. Требуется определить результат при оптимальной игре.

**Решение.** Удобно оптимизировать не собственную сумму игрока отдельно, а разность между его итоговой суммой и суммой противника. Пусть

$$D[l][r] = \max\{\text{сумма текущего игрока} - \text{сумма другого игрока}\},$$

если перед ходом на столе остался подмассив  $a_l, \dots, a_r$ .

Если текущий игрок берёт  $a_l$ , то далее роли игроков меняются. На оставшемся отрезке противник может обеспечить себе преимущество  $D[l+1][r]$ , поэтому преимущество нынешнего игрока равно  $a_l - D[l+1][r]$ . Аналогично для правого конца. Следовательно,

$$D[l][r] = \max(a_l - D[l+1][r], a_r - D[l][r-1]), \quad D[i][i] = a_i.$$

Состояния вычисляются по возрастанию длины отрезка.

Пусть  $S = \sum_{i=1}^n a_i$  и  $D = D[1][n]$ . Тогда суммы первого и второго игроков равны соответственно

$$X = \frac{S + D}{2}, \quad Y = \frac{S - D}{2}.$$

Поэтому первый игрок выигрывает при  $D > 0$ , второй — при  $D < 0$ , а при  $D = 0$  получается ничья. Формулы остаются верными и для отрицательных  $a_i$ .

**Доказательство корректности.** Докажем по длине  $r-l+1$ , что  $D[l][r]$  действительно равно максимальному гарантируемому преимуществу текущего игрока.

Для отрезка длины 1 единственный ход приносит  $a_i$ , а противник ничего не получает, поэтому  $D[i][i] = a_i$ .

Пусть утверждение доказано для всех более коротких отрезков. Первый ход из  $[l, r]$  обязан удалить либо  $a_l$ , либо  $a_r$ . В первом случае, по предположению индукции, ставший текущим противник получает преимущество  $D[l+1][r]$ ; следовательно, преимущество сделавшего первый ход равно  $a_l - D[l+1][r]$ . Во втором случае оно равно  $a_r - D[l][r-1]$ . Игрок выбирает лучший из двух вариантов, что в точности даёт указанную рекуррентную формулу. Других первых ходов нет, поэтому полученное значение и достижимо, и не может быть улучшено.  $\square$

**Сложность.** Состояний  $\Theta(n^2)$ , переход занимает  $\mathcal{O}(1)$ , поэтому время равно  $\mathcal{O}(n^2)$ , память —  $\mathcal{O}(n^2)$ . Если нужна только величина ответа, диагонали таблицы можно хранить послойно и сократить память до  $\mathcal{O}(n)$ .

## 2. Задача 2. Путь с минимальным числом завершающих нулей

**Условие.** В целочисленной матрице нужно пройти из левого верхнего угла в правый нижний, двигаясь только вправо и вниз, и минимизировать число нулей в конце произведения чисел на пути.

**Решение.** Для ненулевого целого  $x$  обозначим через  $\nu_p(x)$  показатель простого  $p$  в разложении  $|x|$ . Число завершающих десятичных нулей произведения  $P \neq 0$  равно

$$\nu_{10}(P) = \min(\nu_2(P), \nu_5(P)).$$

Кроме того, показатели произведения складываются. Поэтому отдельно решим две обычные задачи о пути минимального веса.

Для  $p \in \{2, 5\}$  положим

$$w_p(i, j) = \nu_p(a_{ij})$$

для ненулевых клеток и заведём таблицу

$$d_p[i][j] = w_p(i, j) + \min(d_p[i-1][j], d_p[i][j-1]),$$

где отсутствующему предшественнику приписывается  $+\infty$ , а  $d_p[1][1] = w_p(1, 1)$ . Ответ среди путей без нулевых клеток равен

$$\min(d_2[n][m], d_5[n][m]).$$

Важно, что минимум берётся *после* оптимизации каждого показателя:

$$\min_{\pi} \min(A_{\pi}, B_{\pi}) = \min\left(\min_{\pi} A_{\pi}, \min_{\pi} B_{\pi}\right).$$

Остаётся оговорить нулевые клетки. В классической постановке этой задачи произведению 0 условно приписывают один завершающий ноль. Тогда нулевую клетку запрещаем в обеих динамиках (вес  $+\infty$ ), но, если существует монотонный путь через хотя бы один ноль, дополнительно рассматриваем кандидат 1. Для прямоугольной таблицы любой клетке можно сопоставить такой путь: сначала дойти до неё вправо и вниз, затем теми же ходами дойти до  $(n, m)$ . Поэтому достаточно знать, есть ли в матрице ноль.

Если в условии принято чисто теоретико-числовое соглашение, при котором 0 делится на  $10^q$  для любого  $q$ , путь через ноль имеет бесконечно много завершающих нулей и никогда не улучшает конечный ответ; основная динамика остаётся той же.

**Доказательство корректности.** Рассмотрим фиксированное  $p \in \{2, 5\}$ . Любой путь в клетку  $(i, j)$  приходит либо из  $(i-1, j)$ , либо из  $(i, j-1)$ . После последнего шага к сумме показателей на префиксе пути добавляется ровно  $w_p(i, j)$ . Индукция по  $i + j$  поэтому доказывает, что  $d_p[i][j]$  есть минимальная возможная сумма  $\nu_p$  на пути в эту клетку.

Для каждого ненулевого пути число нулей равно минимуму его двух сумм показателей. Равенство минимумов, выписанное выше, доказывает, что лучший ненулевой путь найден одной из двух динамик. Наконец, при принятом соревновательном соглашении любой путь через ноль имеет стоимость ровно 1, поэтому сравнение с дополнительным кандидатом 1 рассматривает также все такие пути.  $\square$

**Сложность.** Две таблицы имеют  $nm$  состояний и заполняются за  $\mathcal{O}(nm)$  времени. Память равна  $\mathcal{O}(nm)$ , либо  $\mathcal{O}(n)$  при хранении только предыдущей строки. Если требуется восстановить сам путь, хранят предка каждого состояния.

### 3. Задача 3. Удаление внутренних элементов

**Условие.** При удалении внутреннего элемента  $a_k$  платится  $a_k$  умноженное на сумму его текущих соседей. Нужно удалить все элементы, кроме исходных крайних, с минимальным суммарным штрафом.

**Решение.** Пусть  $dp[l][r]$  — минимальная стоимость удаления всех исходных элементов с индексами строго между  $l$  и  $r$ , причём  $a_l$  и  $a_r$  остаются. Если  $r = l + 1$ , удалять нечего и  $dp[l][r] = 0$ .

Рассмотрим элемент  $a_k$ , который удаляется *последним* среди  $a_{l+1}, \dots, a_{r-1}$ . К этому моменту все элементы между  $l$  и  $k$ , а также между  $k$  и  $r$ , уже удалены. Поэтому текущими соседями  $a_k$  являются именно  $a_l$  и  $a_r$ , и последний штраф равен

$$a_k(a_l + a_r).$$

До этого независимо решаются два подотрезка. Получаем

$$dp[l][r] = \min_{l < k < r} (dp[l][k] + dp[k][r] + a_k(a_l + a_r)).$$

Таблица заполняется по возрастанию длины  $r - l$ ; искомое значение равно  $dp[1][n]$ . Для восстановления порядка удалений запоминается минимизирующий индекс  $k$ , рекурсивно выводятся порядки для  $[l, k]$  и  $[k, r]$ , после чего выводится  $k$ .

**Доказательство корректности.** Возьмём оптимальный порядок для интервала  $[l, r]$  и обозначим через  $k$  последний удалённый внутренний индекс. До удаления  $k$  должны исчезнуть все индексы из  $(l, k)$  и  $(k, r)$ . Операции в этих двух множествах не меняют друг друга: пока  $k$  существует, элементы слева от него не могут стать соседями элементов справа от него. Следовательно, цена операций слева не меньше  $dp[l][k]$ , а справа — не меньше  $dp[k][r]$ . Последняя операция стоит  $a_k(a_l + a_r)$ . Значит, стоимость любого порядка не меньше одного из кандидатов рекуррентной формулы.

Обратно, для любого  $k$  можно сначала выполнить оптимальные порядки на  $[l, k]$  и  $[k, r]$  в любом взаимном чередовании, а затем удалить  $k$ . Это допустимый порядок с ценой, равной соответствующему кандидату. Минимум кандидатов поэтому в точности равен оптимуму.  $\square$

**Сложность.** Имеется  $\Theta(n^2)$  интервалов, для каждого перебираются  $\mathcal{O}(n)$  последних элементов. Время  $\mathcal{O}(n^3)$ , память  $\mathcal{O}(n^2)$ .

## 4. Задача 4. Игра с камнями

**Решение.** Позиция называется выигрышной, если игрок, которому предстоит ход, может гарантировать победу, и проигрышной иначе. Пустая куча проигрышна.

Общий динамический алгоритм задаётся формулой

$$win[x] = \bigvee_{\substack{q \in M(x) \\ q \leq x}} \neg win[x - q],$$

где

$$M(x) = \begin{cases} \{1, 2\}, & x \equiv 0 \pmod{3}, \\ \{1, 3\}, & x \equiv 1 \pmod{3}, \\ \{1, 2, 3\}, & x \equiv 2 \pmod{3}. \end{cases}$$

Значения  $win[0], \dots, win[n]$  вычисляются слева направо.

В этой задаче динамику можно свернуть в явный ответ:

позиция с  $x$  камнями проигрышна тогда и только тогда, когда  $x \equiv 0 \pmod{3}$ .

Действительно, из кратного трём числа разрешено взять 1 или 2, то есть все ходы ведут в некрatные трём позиции. Из позиции  $x \equiv 1 \pmod{3}$  можно взять 1, а из позиции  $x \equiv 2 \pmod{3}$  — 2; в обоих случаях получается кратное трём число.

**Доказательство корректности.** Проведём индукцию по  $x$ . Для  $x = 0$  ходов нет, и позиция проигрышна. Пусть классификация верна для всех меньших куч.

Если  $x \equiv 0 \pmod{3}$ , каждый допустимый ход оставляет число с остатком 1 или 2, которое по предположению индукции выигрышно. Поэтому текущий игрок неизбежно передаёт противнику выигрышную позицию и проигрывает. Если  $x \not\equiv 0 \pmod{3}$ , описанный выше ход оставляет кратное трём меньшее число, то есть проигрышную позицию. Такой ход обеспечивает победу.  $\square$

**Сложность.** Прямая динамика работает за  $\mathcal{O}(n)$  времени и  $\mathcal{O}(n)$  памяти ( $\mathcal{O}(1)$  памяти при хранении последних значений). Явная формула даёт  $\mathcal{O}(1)$  времени и памяти.

## 5. Задача 5. Гладкие десятичные числа

**Решение.** Будем считать, что  $n \geq 1$ , а натуральное  $n$ -значное число не может начинаться с нуля. Введём

$$dp[t][d] = \#\{\text{допустимых } t\text{-значных чисел с последней цифрой } d\}.$$

Начальные значения:

$$dp[1][0] = 0, \quad dp[1][d] = 1 \quad (1 \leq d \leq 9).$$

Для  $t \geq 2$

$$dp[t][d] = \sum_{\substack{e=0 \\ l \leq |e-d| \leq r}}^9 dp[t-1][e].$$

Ответ равен  $\sum_{d=0}^9 dp[n][d]$ . Если  $l > r$ , либо интервал  $[l, r]$  не содержит ни одной возможной разности  $0, \dots, 9$ , все переходы автоматически дают ноль.

**Доказательство корректности.** Каждое число, учитываемое в  $dp[t][d]$ , получается единственным образом: нужно удалить его последнюю цифру  $d$ , после чего остаётся допустимое  $(t-1)$ -значное число с некоторой последней цифрой  $e$ , удовлетворяющей  $l \leq |e-d| \leq r$ . Обратно, дописывание  $d$  к любому такому числу сохраняет все старые ограничения и добавляет ровно одно новое, также выполненное. Слагаемые попарно не пересекаются, поэтому переход считает каждое допустимое число ровно один раз. Суммирование по последней цифре покрывает все  $n$ -значные числа.  $\square$

**Сложность.** На каждом из  $n$  слоёв рассматриваются не более  $10^2$  пар цифр, то есть время  $\mathcal{O}(n)$ , память  $\mathcal{O}(1)$  при хранении двух массивов длины 10.

## 6. Задача 6. Максимальная цепочка вложенных кругов

**Решение.** Круг  $i$  с центром  $c_i = (x_i, y_i)$  и радиусом  $r_i$  строго содержится в круге  $j$ , если

$$\|c_i - c_j\| + r_i < r_j.$$

Если касание изнутри считается вложением, знак  $<$  заменяется на  $\leq$ . Условие задачи гарантирует ламинарность: для любой пары круги либо не пересекаются, либо один вложен в другой.

Отсортируем круги по неубыванию радиуса. Пусть  $dp[j]$  — максимальная длина цепочки, внешний круг которой равен  $j$ . Тогда

$$dp[j] = 1 + \max_{\substack{i < j \\ i \text{ вложен в } j}} dp[i],$$

где максимум по пустому множеству равен 0. Ответ —  $\max_j dp[j]$ . Сравнение можно выполнять без извлечения квадратного корня:

$$(x_i - x_j)^2 + (y_i - y_j)^2 < (r_j - r_i)^2,$$

предварительно проверив  $r_i < r_j$ .

**Доказательство корректности.** В любой цепочке с внешним кругом  $j$  непосредственно предшествующий ему круг  $i$  вложен в  $j$  и имеет меньший радиус, поэтому находится раньше в сортировке. Часть цепочки до  $i$  имеет длину не больше  $dp[i]$ , а вся цепочка — не больше  $1 + dp[i]$ . Значит, рекуррентная формула не занижает требуемый верхний предел.

Обратно, если индекс  $i$  достигает максимума, оптимальная цепочка, заканчивающаяся в  $i$ , целиком лежит в  $i$ , а  $i$  лежит в  $j$ . Добавление  $j$  даёт допустимую цепочку длины  $1 + dp[i]$ . Следовательно, каждое вычисленное значение достижимо и оптимально.  $\square$

**Сложность.** Сортировка требует  $\mathcal{O}(n \log n)$ , все проверки пар —  $\mathcal{O}(n^2)$ . Итоговое время  $\mathcal{O}(n^2)$ , память  $\mathcal{O}(n)$ .

## 7. Задача 7. Разбиение скобочной строки

**Условие.** Нужно посчитать число способов разрезать данную строку из круглых скобок на непустые правильные скобочные последовательности.

**Решение.** Обозначим баланс префикса длины  $i$  через

$$b[0] = 0, \quad b[i] = b[i-1] + \begin{cases} 1, & s_i = (, \\ -1, & s_i = ). \end{cases}$$

Подстрока  $s_{j+1} \dots s_i$  является правильной скобочной последовательностью тогда и только тогда, когда

$$b[i] = b[j] \quad \text{и} \quad \min_{j < q \leq i} b[q] \geq b[j]. \quad (1)$$

Пусть  $dp[i]$  — число разбиений префикса  $s_1 \dots s_i$ , причём  $dp[0] = 1$ . Перебираем последнее место разреза:

$$dp[i] = \sum_{\substack{0 \leq j < i \\ s_{j+1} \dots s_i \text{ правильна}}} dp[j].$$

Условие правильности проверяется по (1), просматривая подстроку, либо по заранее вычисленному минимуму; даже прямой двойной цикл занимает  $\mathcal{O}(n^2)$ .

Есть и более короткое следствие. Если вся строка не является правильной, ответ равен 0, поскольку конкатенация правильных последовательностей сама правильна. Если строка правильна, разрез разрешён в точности после каждого собственного ненулевого префикса с балансом 0. Если таких позиций  $z$ , ответ равен  $2^z$ : каждую из них независимо выбираем или не выбираем как место разреза.

**Доказательство корректности.** Критерий (1) стандартен, но докажем его. Равенство балансов означает, что в подстроке одинаковое число открывающих и закрывающих скобок. Неравенство для минимумов означает, что ни в одном её префиксе закрывающих скобок не стало больше, чем открывающих. Это в точности определение правильной скобочной последовательности.

В любом разбиении префикса существует единственный последний блок  $s_{j+1} \dots s_i$ . Предшествующая часть допускает  $dp[j]$  разбиений, а последний блок допустим ровно для индексов из суммы. Поэтому переход считает каждое разбиение один раз и не считает недопустимых. Индукция по  $i$  доказывает правильность всей динамики.  $\square$

**Сложность.** Запрошенный алгоритм работает за  $\mathcal{O}(n^2)$  времени и  $\mathcal{O}(n)$  памяти. Указанное упрощение даёт  $\mathcal{O}(n)$  времени и  $\mathcal{O}(1)$  дополнительной памяти, не считая хранения большого целого ответа.

## 8. Задача 8. Умножение соседних чисел на $-1$

**Решение.** Модули всех элементов неизменны. Обозначим

$$S = \sum_{i=1}^n |a_i|, \quad \mu = \min_i |a_i|.$$

Одна операция меняет знаки ровно у двух позиций. Поэтому, если нулей нет, чётность числа отрицательных элементов является инвариантом.

Итоговый ответ:

$$\begin{cases} S, & \text{если есть ноль или число отрицательных элементов чётно,} \\ S - 2\mu, & \text{иначе.} \end{cases}$$

Осталось обосновать достижимость допустимых наборов знаков. Представим решение над  $\mathbb{F}_2$ . Пусть  $x_i = 1$ , если операция на паре  $(i, i+1)$  выполняется нечётное число раз. Требуемая маска переключений  $y_1, \dots, y_n$  удовлетворяет

$$y_1 = x_1, \quad y_i = x_{i-1} \oplus x_i \quad (1 < i < n), \quad y_n = x_{n-1}.$$

Первые  $n-1$  равенств однозначно задают  $x_i = y_1 \oplus \dots \oplus y_i$ , а последнее выполняется тогда и только тогда, когда  $\bigoplus_i y_i = 0$ , то есть переключается чётное число позиций. Следовательно, можно реализовать любую чётную маску переключений.

Если отрицательных элементов чётное число, переключаем именно их и делаем все числа неотрицательными. Если оно нечётно и нулей нет, чётность сохранится, поэтому хотя бы одно число останется отрицательным; выгоднее оставить отрицательным элемент минимального модуля. Если есть ноль, переключаем все отрицательные позиции и одну нулевую: маска имеет чётную мощность, а знак нуля не влияет на значение.

**Доказательство корректности.** Инвариант показывает, что при нечётном числе отрицательных ненулевых элементов сумма  $S$  недостижима. В любой допустимой конфигурации остаётся хотя бы один отрицательный элемент, а замена  $+|a_i|$  на  $-|a_i|$  уменьшает сумму на  $2|a_i|$ . Значит, сумма не превосходит  $S - 2\mu$ . Конструкция через систему над  $\mathbb{F}_2$  достигает именно этой границы.

В двух остальных случаях та же конструкция делает каждый элемент неотрицательным, достигая  $S$ . Больше суммы быть не может, поскольку каждый отдельный элемент не превосходит собственного модуля.  $\square$

**Сложность.** Один проход по массиву:  $\mathcal{O}(n)$  времени и  $\mathcal{O}(1)$  памяти.

## 9. Задача 9. Разложения целого числа

### Пункт Неупорядоченные разложения

**Решение.** Пусть слагаемые разрешено брать из множества  $\{1, \dots, n\}$ . Заведём одномерную динамику  $dp[s]$ , равную числу способов набрать сумму  $s$  уже рассмотренными размерами слагаемых. Инициализация  $dp[0] = 1$ ,  $dp[s] = 0$  для  $s > 0$ . Для каждого  $q = 1, \dots, n$  выполняем

$$\text{для } s = q, q + 1, \dots, n : \quad dp[s] \leftarrow dp[s] + dp[s - q].$$

Возрастание  $s$  позволяет использовать  $q$  любое число раз. Внешний цикл по  $q$  гарантирует, что одна и та же мультимножество слагаемых не пересчитывается в разных порядках. Ответ —  $dp[n]$ .

**Доказательство корректности.** После обработки чисел  $1, \dots, q$  докажем, что  $dp[s]$  равно числу разложений  $s$  на части не больше  $q$ . Любое такое разложение либо не содержит  $q$ , либо после удаления одного  $q$  превращается в разложение  $s - q$  на части не больше  $q$ . Эти классы не пересекаются и исчерпывают все варианты, а присваивание реализует именно это разбиение. База  $dp[0] = 1$  соответствует пустому разложению. Индукция по  $q$  завершает доказательство.  $\square$

### Пункт Упорядоченные разложения

Упорядоченное разложение, или композиция, числа  $n \geq 1$  взаимно однозначно соответствует выбору разрезов в  $n - 1$  промежутках между единицами

$$1 \mid 1 \mid \dots \mid 1.$$

Разрез означает конец очередного слагаемого. Каждый промежуток независимо либо разрезан, либо нет, поэтому число композиций равно

$$\boxed{2^{n-1}}.$$

Эквивалентная динамика:

$$c[0] = 1, \quad c[s] = \sum_{q=1}^s c[s - q],$$

из которой по индукции также получается  $c[s] = 2^{s-1}$  для  $s \geq 1$ .

**Сложность.** Неупорядоченные разложения считаются за  $\mathcal{O}(n^2)$  времени и  $\mathcal{O}(n)$  памяти. Для упорядоченных ответ вычисляется быстрым возведением двойки в степень за  $\mathcal{O}(\log n)$  арифметических операций ( $\mathcal{O}(n)$ , если число строится последовательно).

## 10. Задача 10. Максимальный квадрат из единиц

**Решение.** Для каждой клетки определим  $dp[i][j]$  как сторону максимального квадрата из единиц, правый нижний угол которого находится в  $(i, j)$ . Если  $a_{ij} = 0$ , то  $dp[i][j] = 0$ . Если  $a_{ij} = 1$ , то

$$dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]), \quad (2)$$

где значения вне таблицы считаются нулевыми. Максимум всех  $dp[i][j]$  есть искомая сторона; сам квадрат восстанавливается по клетке, где достигнут максимум.

**Доказательство корректности.** Пусть  $a_{ij} = 1$  и минимум в (2) равен  $q$ . Три соседних значения гарантируют наличие:

- квадрата  $q \times q$ , заканчивающегося сверху от  $(i, j)$ ;
- квадрата  $q \times q$ , заканчивающегося слева;
- квадрата  $q \times q$ , заканчивающегося по диагонали.

Их объединение с клеткой  $(i, j)$  покрывает квадрат  $(q+1) \times (q+1)$  из единиц, поэтому значение  $q+1$  достижимо.

Обратно, если существует квадрат стороны  $q+1$  с правым нижним углом  $(i, j)$ , удаление его нижней строки, правого столбца либо одновременно обоих даёт три квадрата стороны  $q$  с соответствующими правыми нижними углами. Значит, каждое из трёх соседних значений не меньше  $q$ , и больший ответ возможен только тогда, когда формула также его допускает. Поэтому  $dp[i][j]$  вычислено точно.  $\square$

**Сложность.**  $n^2$  клеток обрабатываются за  $\mathcal{O}(1)$ : время  $\mathcal{O}(n^2)$ , память  $\mathcal{O}(n^2)$ , либо  $\mathcal{O}(n)$  при двух строках.

## 11. Задача 11. Удаление каждого $k$ -го элемента по раундам

**Уточнение формулировки.** Пример при  $n = 10, k = 2$  заканчивается на 9 и оставляет 1. Следовательно, раунды прекращаются, когда текущая длина меньше  $k$ ; последние  $k - 1$  элементов не удаляются. Ниже решается именно эта интерпретация, согласованная с примером. При  $k = 1$  удаляются все элементы в первом раунде.

**Решение.** Для исходного индекса  $x$  вычислим номер раунда  $\text{round}[x]$ , в котором он исчезнет; значение  $\infty$  означает, что он останется.

В первом раунде удаляются ровно индексы, кратные  $k$ . Если  $x$  не кратен  $k$ , то после первого раунда его новый порядковый номер равен

$$T(x) = x - \left\lfloor \frac{x}{k} \right\rfloor, \quad (3)$$

потому что среди первых  $x$  позиций удалено  $\lfloor x/k \rfloor$ . Отсюда

$$\text{round}[x] = \begin{cases} 1, & k \mid x, \\ \infty, & x < k, \\ 1 + \text{round}[T(x)], & x \geq k, k \nmid x. \end{cases} \quad (4)$$

При  $x \geq k$  имеем  $T(x) < x$ , поэтому значения (4) вычисляются в порядке  $x = 1, \dots, n$ , за постоянное время каждое.

Создадим списки  $\text{bucket}[r]$ . Каждый удаляемый индекс  $x$  добавим в  $\text{bucket}[\text{round}[x]]$ , просматривая  $x$  по возрастанию. Ответом служит конкатенация  $\text{bucket}[1], \text{bucket}[2], \dots$ . Внутри одного раунда исходный порядок сохраняется, поэтому сортировка не нужна.

**Доказательство корректности.** Докажем формулу (4). Кратные  $k$  удаляются в первом раунде по определению. Индексы  $1, \dots, k - 1$  никогда не удаляются: пока они существуют, они остаются первыми  $k - 1$  элементами, а раунд удаляет только позиции  $k, 2k, \dots$

Пусть  $x \geq k$  и  $k \nmid x$ . Он переживает первый раунд, и (3) точно задаёт его позицию в оставшемся массиве. Дальнейший процесс над оставшимся массивом совпадает с исходным процессом, только номера раундов сдвинуты на один. Следовательно,  $x$  удаляется через  $1 + \text{round}[T(x)]$  раундов либо не удаляется вовсе. Это доказывает (4) индукцией по  $x$ .

В каждом раунде элементы удаляются слева направо, то есть в порядке возрастания исходных индексов: удаление предыдущих раундов не меняет относительный порядок выживших. Поэтому разложение по корзинам выдаёт точный глобальный порядок удаления.  $\square$

**Сложность.** Каждый индекс обрабатывается один раз. Время  $\mathcal{O}(n)$ , память  $\mathcal{O}(n)$ . Число непустых корзин не превосходит  $n$ .

## 12. Задача 12. Теорема Эрдёша-Секереша

**Решение.** Пусть дана перестановка  $a_1, \dots, a_{n^2+1}$ . Для каждого индекса  $i$  обозначим через  $I_i$  длину наибольшей возрастающей подпоследовательности, заканчивающейся в  $a_i$ , а через  $D_i$  — длину наибольшей убывающей подпоследовательности, заканчивающейся в  $a_i$ . Предположим противное: ни возрастающей, ни убывающей подпоследовательности длины  $n + 1$  нет. Тогда

$$(I_i, D_i) \in \{1, \dots, n\}^2,$$

то есть возможны лишь  $n^2$  пар.

Покажем, что пары для различных индексов различны. Если  $i < j$ , то из свойства перестановки  $a_i \neq a_j$ . При  $a_i < a_j$  к возрастающей подпоследовательности, заканчивающейся в  $a_i$ , можно дописать  $a_j$ ; следовательно,  $I_j \geq I_i + 1$ . При  $a_i > a_j$  аналогично  $D_j \geq D_i + 1$ . В обоих случаях  $(I_i, D_i) \neq (I_j, D_j)$ .

Итак,  $n^2 + 1$  индексам сопоставлены попарно различные элементы  $n^2$ -элементного множества. Это противоречит принципу Дирихле. Следовательно, хотя бы одно из чисел  $I_i, D_i$  не меньше  $n + 1$ , что и даёт требуемую подпоследовательность.

**Доказательство корректности.** В рассуждении одновременно доказаны две необходимые части: если все монотонные подпоследовательности короче  $n + 1$ , каждая пара лежит в  $n \times n$  клетках; и никакие два индекса не могут занять одну клетку. Принцип Дирихле делает эти утверждения несовместимыми для  $n^2 + 1$  индексов. Полученное противоречие доказывает теорему.  $\square$

**Сложность.** Это комбинаторное доказательство. Если требуется конструктивно найти одну из подпоследовательностей, значения  $I_i, D_i$  с предками можно вычислить за  $\mathcal{O}(n^2)$  прямой динамикой или за  $\mathcal{O}(n \log n)$  стандартным алгоритмом поиска наибольшей возрастающей подпоследовательности.